CS 6999: Introduction to Reinforcement Learning

Assignment 3: Monte Carlo Methods

Student Name: Arefeh Pourmohammadi Student Number: 3803638

[**Mandatory**] Declaration: "I warrant that this is my own work."

Signed by Arefeh Pourmohammadi (You can type in your name as your signature.)

Leave the table blank. For marking only.

Achieved	**Late Penalty**	**Others (Specify)**	**Others (Specify)**	**Total Score**
				/
Additional comments if any:				

From the UNB Graduate Calendar, available at website `https://www.unb.ca/gradstudies/current/resources/regulations-and-guidelines/regulations/academic-offenses.html`: "Plagiarism includes:

(a) quoting verbatim or almost verbatim from a source (such as copyrighted material, notes, letters, business entries, computer materials, etc.) without acknowledgment;

(b) adopting someone else's line of thought, argument, arrangement, or supporting evidence (such as, for example, statistics, bibliographies, etc.) without indicating such dependence;

(c) submitting someone else's work, in whatever form (film, workbook, artwork, computer materials, etc.) without acknowledgment;

(d) knowingly representing as one's own work any idea of another;

(e) contravention of written instructions of the instructor dealing with plagiarism."

Other academic offences are also set out in the Graduate Calendar. Penalties for plagiarism and other academic offences are also given at the above website.

Auxiliary Files

In the `play_game` file, the dealer's hidden card is first created randomly using the code:

```
min(random.randint(1, 13), 10)
```

This is because four cards (the face cards and the 10) have a score of 10, while the other cards have their own face values. After creating this card, I check if either of the dealer's cards is an ace to count it as 11, and then the game begins.

The initial state is received from the control policy. For example, in MCES, the first state and action are sampled from all possible state-action pairs. In the on-policy and off-policy methods, the first state and action are generated when the player's sum is greater than 11 and the obvious "hit" action is no longer required. The policy (or behavior policy) suggests an action for that state, which is then sent to the `play_game` file. The player then acts according to the policy until the game ends. The game is finished if either the player or dealer exceeds 21, or if the player sticks and the dealer's sum is greater than 17. The reward is defined as requested in the problem.

For the MDP state transition, if the action is hit, a new random card is added to the player's total sum. Otherwise, the dealer draws cards randomly until their hidden sum exceeds 17. Note that if the state is a natural blackjack (an ace and a ten-value card) in the first iteration of the loop in `play_game`, it is handled accordingly.

Control Policies

I have implemented three control policies: exploring starts (MCES), on-policy MC with an ϵ-greedy policy, and off-policy MC. For the off-policy method, the behavior policy is by default the equiprobable policy, but it can be changed. All of these policies are implemented based on the pseudocode from the book.

For the on-policy and off-policy methods, I start the game from the beginning where the player receives two cards and continues to hit until the sum is less than 12. This part is not strictly necessary, as one could start from different states, but I chose this approach to avoid making it like the exploring starts method.

Another important point is the return computation. Instead of averaging the return each time with $O(n)$ complexity, I use incremental averaging with the help of `num.returns` and `sum.returns`, making it $O(1)$. However, I still append G to the returns list to keep it familiar to the pseudocode, even though the returns list itself is not used elsewhere in my implementation.

The output of the policies is shown in the following figures. The MCES result is obtained after 5,000,000 episodes, like the result in the book, while the on-policy and off-policy results are obtained after 50,000,000 episodes. The figures below show the MCES optimal policy and value function, followed by the on-policy and off-policy results, respectively.

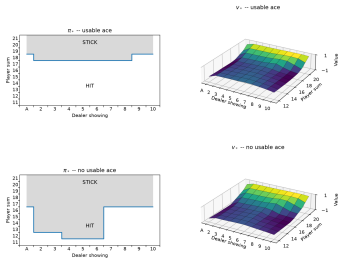

Figure 1: MCES optimal policy and value function

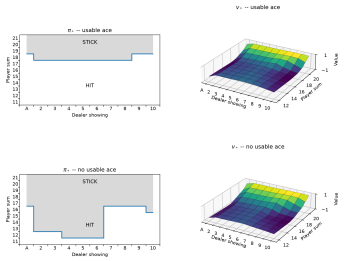

Figure 2: On-Policy optimal policy and value function

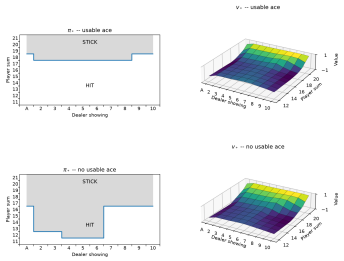

Figure 3: Off-Policy optimal policy and value function

Policy Prediction

As described in the book, there are two policy prediction methods: one with exploring first-visit and the other with off-policy. Both of them are implemented based on the book's pseudocode and using the auxiliary files. The results are shown below, which are similar to the book's output. Both methods were executed once with 500,000 episodes and once with 10,000 episodes to compare their results.

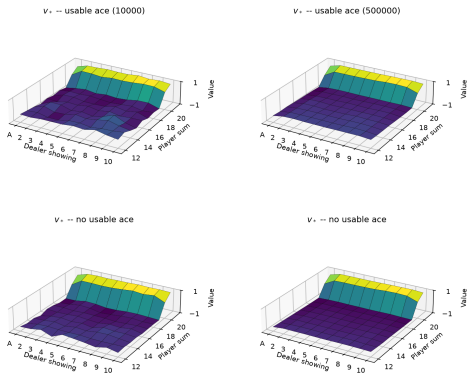

Figure 4: Exploring start prediction

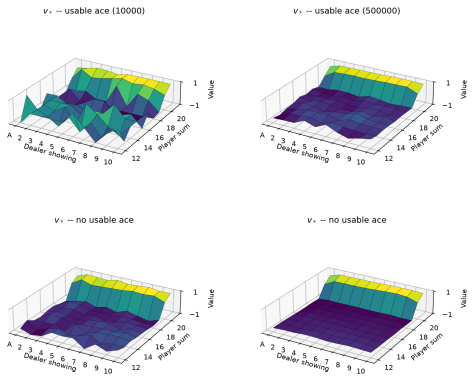

Figure 5: Off-policy prediction